Miniexamen 9

Curso de Python

Nombre: _____

Resolver todo en un script y responder las preguntas como comentarios dentro del mismo script.

```
import pandas as pd
import numpy as np
import seaborn as sns
```

Ejercicio 1. Series y DataFrame

Los dos objetos básicos de pandas son la **Series** (1D: una columna de datos con su *index*) y el **DataFrame** (2D: varias columnas que comparten el mismo index).

1.1 Crear Series

```
s1 = pd.Series([1, 2, 3, 4, 5])
s2 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
s3 = pd.Series([10, 20, 30, 40, 50])
```

a) Imprimir `s1`, `s2` y el tipo de `s1` con `type()`.

b) ¿Qué index le asignó pandas a `s1`, si no se le pasó ninguno?

1.2 Operaciones elemento a elemento

Las operaciones aritméticas con Series se aplican elemento a elemento, igual que con arreglos de numpy.

a) Calcular `s1 * 2`, `s1 + 5` y `s1 ** 2`.

b) Calcular `s1 + s3` y `s1 * s3`.

c) Calcular `s1 * s2`. ¿Qué resultado da? ¿Por qué, si `s1` y `s2` tienen exactamente los mismos valores?

d) Crear dos Series con index distinto y sumarlas:

```
s4 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s5 = pd.Series([10, 20, 30], index=['b', 'c', 'd'])
print(s4 + s5)
```

¿Qué pasa con las etiquetas que no coinciden? Este comportamiento se llama *alineación por index*; explicarlo con palabras propias.

1.3 Funciones de numpy sobre Series

a) Aplicar `np.sqrt`, `np.exp` y `np.log` a `s1`. Por ejemplo: `print(np.sqrt(s1))`.

b) ¿Qué tipo de objeto regresan? ¿Con qué index? Probar también `np.sqrt(s2)`.

c) ¿Por qué se usan funciones de numpy y no de pandas para esto?

1.4 Estadística básica

Las operaciones de 1.2 y 1.3 regresan una Series del mismo tamaño que la original (un resultado por elemento). Las de esta sección, en cambio, *reducen* la Series a un solo número.

a) Calcular `s1.mean()`, `s1.sum()`, `s1.std()`, `s1.min()` y `s1.max()`.

b) Aplicar `s1.describe()`. ¿Qué regresa?

c) Comparar `s1.std()` con `np.std(s1.values)`. ¿Dan lo mismo? Buscar en la documentación qué hace el parámetro `ddof` en cada una.

1.5 Un primer DataFrame

```
df_chico = pd.DataFrame({
    'nombre': ['Ana', 'Beto', 'Carla'],
    'edad':   [25, 30, 35],
    'altura': [1.60, 1.75, 1.68]
})
```

a) Imprimir `df_chico`, su `index` y sus tipos de datos con `.dtypes`.

b) Extraer la columna `'edad'` con `df_chico['edad']`. ¿De qué tipo es?

c) Renombrar la columna `'edad'` como `'anios'`:

```
df_renombrado = df_chico.rename(columns={'edad': 'anios'})
```

Imprimir `df_renombrado` y luego `df_chico`. ¿Cambió el original?

d) Eliminar la columna `'altura'` con `df_chico.drop(columns=['altura'])`.

e) Eliminar la fila de Beto con `df_chico.drop(index=1)`.

f) Convertir la columna `'edad'` a `float` con `.astype(float)` y verificar el cambio con `.dtypes`.

Ejercicio 2. Análisis del dataset `tips`

El dataset `tips` contiene información de las cuentas y propinas de un restaurante.

```
tips = sns.load_dataset('tips')
```

2.1 Exploración inicial

a) Imprimir las primeras 5 filas con `head()`, las últimas 5 con `tail()` y las primeras 10 (`head` acepta un argumento).

b) Usar `.info()` para ver los tipos de datos y si hay valores nulos.

c) Usar `.describe()`. ¿Qué columnas no aparecen? ¿Por qué?

d) Imprimir `.shape`, `.columns` e `.index`. ¿Cuántas filas y columnas tiene?

2.2 Selección de columnas

a) Imprimir solo la columna `'tip'`. ¿Qué tipo de objeto es?

b) Imprimir la columna `'tip'` como DataFrame (doble corchete). ¿Qué tipo es ahora?

c) Imprimir `'tip'` y `'total_bill'` juntas, pasando una lista con los dos nombres.

2.3 Selección de filas con `loc` e `iloc`

a) Acceder a la primera fila con `iloc` y con `loc`. ¿Se ven iguales? ¿Por qué? (Revisar el index de `tips`.)

b) Ordenar `tips` por propina, de mayor a menor:

```
tips_ordenado = tips.sort_values('tip', ascending=False)
print(tips_ordenado.head())
```

El index ya no es `0, 1, 2, ...`. Comparar `tips_ordenado.iloc[0]` con `tips_ordenado.loc[0]`. ¿Qué fila da cada uno?

c) Explicar con palabras propias la diferencia entre `loc` e `iloc`.

d) Sobre `tips`, obtener las primeras filas con `tips.loc[0:5]` y con `tips.iloc[0:5]`. ¿Por qué una da 6 filas y la otra 5?

e) Probar `tips_ordenado.loc[0:5]`. ¿Qué pasa y por qué? (`loc` toma el rango de etiquetas en el orden en que aparecen en el index.)

A partir de aquí se trabaja otra vez con `tips`, no con `tips_ordenado`.

f) Acceder a una sola celda, la fila con etiqueta 10 y la columna `'tip'`, con `tips.loc[fila, columna]`. Obtener la misma celda con `iloc` (la columna `'tip'` está en la posición 1).

g) Obtener las filas 5 a 10 y las columnas `'tip'` y `'total_bill'` con `loc`. Obtener exactamente lo mismo con `iloc`.

2.4 Filtros (máscaras booleanas)

a) Crear una máscara booleana para las cuentas mayores a 20 dólares, imprimirla y usarla para filtrar `tips`.

b) En una sola línea, filtrar las filas donde: la propina sea mayor a 5; el día sea `'Sun'`; haya más de 4 personas (`size > 4`).

c) Combinar condiciones con `&`, poniendo cada una entre paréntesis, `(cond1) & (cond2)`, para obtener las cuentas mayores a 30 *y* con propina mayor a 5.

d) Filtrar las filas donde el día sea `'Sun'` *o* `'Sat'` usando `|`. Repetir el filtro con `.isin(['Sun', 'Sat'])`.

e) Repetir el filtro del inciso (a) con `tips.query('total_bill > 20')`.

2.5 Valores únicos y conteos

a) Para la columna `'day'`: obtener sus valores únicos con `.unique()`, cuántos son con `.nunique()` y cuántas veces aparece cada uno con `.value_counts()`.

b) Repetir para `'time'` y `'sex'`.

2.6 Operaciones y nuevas columnas

a) Crear la columna `'tip_pct'` con el porcentaje de propina: `tip / total_bill * 100`.

b) Crear la columna `'tip_per_person' = tip / size`.

c) Crear una columna con `.assign()`:

```
tips_nuevo = tips.assign(propina_grande = tips['tip'] > 5)
```

Imprimir `tips_nuevo` y luego `tips`. ¿Cuál es la diferencia con los incisos (a) y (b)?

d) Usar `assign` con una `lambda`:

```
tips.assign(ratio = lambda x: x['tip'] / x['total_bill'])
```

e) Ejecutar el siguiente bloque encadenado. Dentro de cada `lambda`, `x` es el DataFrame *en ese momento* (ya filtrado, ya con `ratio`):

```
resultado = (tips
    .query('total_bill > 20')
    .assign(ratio = lambda x: x['tip'] / x['total_bill'])
    .assign(buena_propina = lambda x: x['ratio'] > 0.15)
)
print(resultado.head())
```

Reescribir lo mismo *sin* `lambda`. ¿Cuántos pasos y variables intermedias se necesitan? ¿Por qué?

2.7 apply y funciones

a) Crear la columna `'categoria_propina'` usando `apply`: `'baja'` si `tip < 2`, `'media'` si `tip < 5` y `'alta'` en otro caso.

```
def clasificar(x):
    if x < 2:
        return 'baja'
    elif x < 5:
        return 'media'
    else:
        return 'alta'

tips['categoria_propina'] = tips['tip'].apply(clasificar)
```

b) Hacer lo mismo con una `lambda` en una sola línea.

c) Usar `apply` sobre filas completas (`axis=1`) para crear una columna que diga `'caro'` si `total_bill > 30` y `size > 3`, y `'normal'` en otro caso.

2.8 Ordenar

a) Ordenar `tips` por `'tip'` de mayor a menor con `sort_values(by='tip', ascending=False)`.

b) Ordenar por dos columnas: primero `'day'` y luego `'tip'`.

c) Ordenar `tips_ordenado` por su index con `sort_index()`. ¿Qué pasa? ¿Por qué con `tips` no se notaría?

2.9 groupby

`groupby` no calcula nada hasta que se le pide algo. Primero, revisar qué hay dentro:

```
g = tips.groupby('day', observed=True)
print(g)                    # <DataFrameGroupBy object at ...>
print(g.groups)             # {dia: [indices de las filas de ese dia]}
print(g.size())             # cuantas filas hay en cada grupo
print(g.get_group('Sun'))   # solo las filas del domingo

for nombre, grupo in g:
    print(f'--- {nombre} ---')
    print(grupo.head(2))
```

a) Calcular la propina promedio por día: `tips.groupby('day', observed=True)['tip'].mean()`.

b) Agrupar por `'sex'` y obtener la cuenta promedio (`total_bill`).

c) Agrupar por `'day'` y calcular varias estadísticas de `'tip'` con `.agg(['mean', 'std', 'count'])`.

d) Agrupar por `'day'` y `'time'`. ¿Qué estructura tiene el index del resultado?

e) Aplicar funciones distintas a columnas distintas:

```
tips.groupby('day', observed=True).agg(
    {'tip': 'mean', 'total_bill': 'sum', 'size': 'max'})
```

2.10 Pivot tables

a) Crear una pivot table con `'day'` como index, `'time'` como columnas y el promedio de `'tip'` como valor:

```
tips.pivot_table(index='day', columns='time', values='tip',
                 aggfunc='mean', observed=True)
```

b) Crear otra con `'sex'` en el index, `'smoker'` en las columnas, sumando `total_bill`.

c) ¿En qué se diferencia `pivot_table` de `groupby`? Comparar el resultado de (a) con el de 2.9(d).